LegacyDesk: Render-only demonstration

Recording guide | 18 October 2026

All demonstration websites run on Render. Use a browser and its DevTools; no local server, Docker stack, command-line probe or traffic generator is needed.

1. Prepare before recording

Purpose	Address
V1 original portal	https://legacydesk-v1-demo.onrender.com
V2 hardened portal	https://legacydesk-v2-demo.onrender.com
Separate CSRF form	https://legacydesk-csrf-demo.onrender.com

1. Open both portals before recording and wait for them to wake.
2. Sign in to V1 using **alice / alice123**. All portal accounts and data are fictional.
3. Sign in to V2 using **alice** and its separate password from PORTAL_ALICE_PASSWORD in Render's Environment page. Retrieve it privately before recording; never show it or real V2 session tokens.
4. Keep both account pages open in the same browser profile as the CSRF form. A separate private window is needed only for the V2 forged-cookie check.
5. Open DevTools -> Network, enable Preserve log, and clear the list before each scene. Use the request's Status column for HTTP results.
6. Use the sequence: **V1 result -> explain protection -> V2 result -> legitimate action**.

V2 credentials: https://dashboard.render.com/web/srv-db9oi73ncjis73a3m9r0/env

What this hosted recording proves

Finding	Honest hosted scope
V02-V07	Six real hosted comparisons. The external CSRF form was rehearsed in the browser on 18 October; earlier hosted checks covered the other findings. Rehearse them before recording.
V01 SQL injection	Previously filtered by Render's upstream Cloudflare protection. Do not claim a successful hosted V1 bypass or a V1 application fix. Omit from the six-finding comparison.
V08 availability	Excluded at your request. No load test, outage scene or simulated result.

Free-service data and sessions can reset after sleep, restart or redeploy. Check sign-in and fictional data before each take. Reset a changed email through the portal's normal account form when needed.

Opening line: "These are two separate Render deployments of the same employee portal. We will compare six real security findings using fictional data. SQL injection is filtered by the hosting platform, and availability testing is excluded."

2. V02 - Stored cross-site scripting

V1: show browser interpretation of stored input

1. Confirm you are Alice and open:

```
https://legacydesk-v1-demo.onrender.com/ticket?id=1001
```

2. Paste this harmless payload into Add a comment and click Post comment:

```
<svg onload="this.outerHTML='<p>XSS executed</p>'"></svg>
```

3. Reload the ticket. Show **XSS executed** in the conversation. The input was stored, then interpreted by the browser.

V2: show rejection and normal use

1. Open https://legacydesk-v2-demo.onrender.com/ticket?id=1001
2. Submit the same payload through the normal comment form, while signed in as Alice.
3. Show the rejected request in Network. The earlier hosted result was HTTP 403 from the WAF.
4. Return to ticket 1001 and post **Normal demonstration comment**. Show that ordinary comments work.

Say: "V1 renders stored input as browser code. V2 combines output encoding, a content security policy and WAF inspection. This hosted rejection demonstrates the WAF layer; it does not separately prove the output-encoding layer."

Optional evidence: search V2 Render logs for **941100** after submitting the payload. Do not expose credentials or session values in logs.

3. V03 - Broken ticket ownership checks

1. Show Alice's identity on both portals.
2. On V1, open Alice's ticket 1001, then change only the identifier:

```
https://legacydesk-v1-demo.onrender.com/ticket?id=1002
```

3. Show Bob's ticket details while the header still identifies Alice.
4. Repeat on V2:

```
https://legacydesk-v2-demo.onrender.com/ticket?id=1002
```

5. Show Access unavailable and HTTP 403.
6. Return to V2 ticket 1001 to prove Alice can still read her own ticket.

Say: "Being signed in does not grant access to every ticket. V2 checks the selected ticket's owner or administrator permission before returning its contents."

4. V04 - Forged identity cookie

Use only the named fictional lab accounts. Keep genuine V2 session values hidden.

V1: change the client-controlled identity

1. While signed in as Alice, open:

```
https://legacydesk-v1-demo.onrender.com/admin
```

2. Show the initial denial: Alice is not an administrator.
3. Open DevTools -> Application -> Storage -> Cookies -> the V1 origin. In Firefox, use Storage -> Cookies.
4. Find the **user_id** cookie. Change its value from **2** to **1**, then commit the edit with Enter.
5. Reload /admin. Show the administrator page and identity.
6. Restore user_id to 2, then refresh. Alternatively, sign out and sign in as Alice again before continuing.

Say: "V1 trusts a user number supplied by the browser. Changing it makes the application treat the visitor as a different account."

V2: repeat with only the fictional forged cookie

1. Open a new private browser window with no existing V2 session.
2. Visit https://legacydesk-v2-demo.onrender.com/login to establish the site's storage entry. Do not sign in in this window.
3. Open DevTools -> Application -> Cookies -> the V2 origin. Add a cookie with these values:

Cookie field	Value
Name	user_id
Value	1
Domain	legacydesk-v2-demo.onrender.com
Path	/
Secure	Enabled

4. Open https://legacydesk-v2-demo.onrender.com/admin
5. Show that no administrator access is granted. The earlier hosted request containing only this fake cookie returned HTTP 401.
6. Close the private window. In the original authenticated Alice window, visit V2 /admin. Expect HTTP 403: Alice is signed in but lacks administrator permission.
7. Open Alice's V2 ticket 1001 to show normal authenticated use still works.

Say: "A made-up user_id cookie does not establish a V2 session. V2 uses an opaque server-validated session, then separately checks administrator permission."

If the browser does not let you create a cookie row, do not substitute an edited genuine V2 session token. Record the genuine Alice denial and identify the fake-cookie check as previously verified rather than freshly demonstrated.

5. V05 - Cross-site request forgery

Everything in this scene is hosted on Render. The third website supplies an ordinary external form; each button makes one request only when clicked. It holds no passwords or session tokens.

V1: demonstrate the unauthorized account change

1. Sign in as Alice and open:

```
https://legacydesk-v1-demo.onrender.com/account
```

2. Show and note the existing email. Keep this tab open.
3. Open a second tab in the same browser profile:

```
https://legacydesk-csrf-demo.onrender.com
```

4. Under V1 / Vulnerable, enter **csrf.take1@example.test**, or another fictional email different from the current account value.
5. Click **Submit external form to V1**. A new result tab opens.
6. Show **Account details updated**. Return to the original V1 account tab and refresh.
7. Show that the email now matches the external form value. This persisted change is the evidence; a button click alone is insufficient.

V2: repeat and verify unchanged state

1. In the original profile, confirm Alice is signed in to V2. Open its /account page and note the current email.
2. Return to the external form. Under V2 / Patched, enter **csrf.blocked@example.test**, different from V2's current value.
3. Click **Submit external form to V2**.
4. Show **Access unavailable** in the new response tab. Check Network for the rejected POST; expected HTTP 403.
5. Refresh the original V2 account tab. Show that its email remains unchanged.
6. Make a normal account update from V2's own form and show that it succeeds. Use a fictional email and restore the earlier value if desired.

Say: "The outside form changes V1's account using the browser's existing authentication. V2 rejects the same outside request. Its Strict session cookie, allowed-Origin validation and session-bound CSRF token provide protection."

Evidence boundary: the browser may withhold V2's Strict cookie on the external POST. This browser scene shows the combined protection, not an isolated token check. Earlier hosted request checks also tested a supplied valid session with an invalid Origin and no CSRF token.

Rehearsal on 18 October: the Render form saved csrf.hosted.v1@example.test on V1. V2 displayed Access unavailable and retained alice.updated@example.test instead of the attempted csrf.hosted.v2@example.test. Freshly refresh both account pages during your recording. Browser privacy settings and expired sessions can change the outcome; do not claim a V1 success if it redirects to sign-in.

6. V06 - Path traversal

1. On V1, open the Knowledge base and download the normal handbook.
2. Then open this fixed lab-fixture path:

```
https://legacydesk-v1-demo.onrender.com/download?file=../private/legacy-config.txt
```

3. Show the returned dummy configuration fixture. Its contents are fictional.
4. While signed in as Alice, repeat on V2:

```
https://legacydesk-v2-demo.onrender.com/download?file=../private/legacy-config.txt
```

5. Show HTTP 403 or 404, without the configuration contents.
6. Open the normal V2 handbook through its Knowledge base to prove allowed downloads still work.

Say: "V1 lets a supplied path escape the download directory. V2 validates the requested resource and restricts file access to the permitted download location."

7. V07 - Debug exposure and response hardening

1. Open https://legacydesk-v1-demo.onrender.com/debug
2. Show the exposed configuration and fictional lab-account information. Do not treat these as real organizational credentials.
3. Repeat at https://legacydesk-v2-demo.onrender.com/debug
4. Show HTTP 404 and no diagnostic contents.
5. Open a normal V2 page. In DevTools -> Network, select its document request -> Headers -> Response Headers.
6. Show the Content-Security-Policy and X-Content-Type-Options headers. Compare the V1 normal-page response if needed; provider-added headers are separate from application protections.

Say: "V2 removes the public debug endpoint and sets browser security headers. A normal page remains available."

Recording roles and closing statement

For four speakers: member 1 opens and shows V02; member 2 shows V03-V04; member 3 shows V05-V06; member 4 shows V07 and closes. Pause between scenes to clear Network and check the current identity. Do not force this six-finding hosted recording into an eight-finding claim.

Closing line: "The hosted comparison shows six real findings and their protections. SQL injection is filtered upstream, so we have not claimed a hosted V1 bypass. V08 is excluded. Legitimate portal actions continue to work."

Hosting references

Render permits testing your own services but prohibits denial-of-service testing: https://render.com/docs/penetration-testing

Render static-site hosting: https://render.com/docs/static-sites

Free-service limits and storage behavior: https://render.com/docs/free

The external form and this guide add no load-test functionality. Original full-lab reports remain historical records; use this guide for the Render-only recording.